

DEV_IMPL_GUIDE - SUB-WF-FRAMEWORK-01-WORKERS

1. Purpose

This guide explains worker implementation as tasks execute at runtime.

Use it with DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md.

2. Runtime Actors

Runtime component	What it does
Camunda	Creates external tasks from BPMN service tasks.
Worker app	Polls Camunda topics, routes tasks to handlers, completes/fails tasks.
Module API clients	Call SUB-LM, BIL, FUL, OSR, NOT, Drools.
Message correlator	Converts external events into Camunda messages.

One worker app may handle many logical topics.

3. Before Workers Can Run

Worker app config example:

```
workerApp: subscription-state-worker
camundaBaseUrl: https://camunda.internal.sophix/engine-rest
workerIdPrefix: subscription-state-worker
topics:
  - sub-lm-put-pending-status
  - sub-lm-commit-state
  - sub-lm-revert-to-prior
  - sub-lm-put-master-fields
  - subscription-operation-finalize
lockDurationMs: 30000
maxTasks: 10
```

Secrets required:

- Camunda internal auth credentials if enabled.
- Service token/client secret for downstream module calls.

4. Flow 1 - Worker App Starts

Example Laravel command:

```
php artisan camunda:worker subscription-state-worker
```

Example Java/container entrypoint:

```
java -jar subscription-state-worker.jar
```

The process:

1. Loads configured topics.
2. Registers one handler per topic.
3. Starts polling loop.
4. Emits heartbeat/ready metric.

5. Flow 2 - Worker Fetches Task From Camunda

Request:

```
POST /engine-rest/external-task/fetchAndLock HTTP/1.1
Content-Type: application/json

{
  "workerId": "subscription-state-worker-01",
  "maxTasks": 10,
  "topics": [
```

```

    {
      "topicName": "sub-lm-put-pending-status",
      "lockDuration": 30000
    },
    {
      "topicName": "sub-lm-commit-state",
      "lockDuration": 30000
    }
  ]
}

```

Camunda returns tasks:

```

[
  {
    "id": "ext_01HZ_PENDING",
    "topicName": "sub-lm-put-pending-status",
    "processInstanceId": "pi_01HZ_NEW_ACT",
    "variables": {
      "operationId": {"value": "subop_01HZ_NEW_ACT", "type": "String"},
      "subscriptionId": {"value": "sub_01HZ_NEW", "type": "String"},
      "toStatus": {"value": "PENDING_ACTIVATION", "type": "String"}
    }
  }
]

```

6. Flow 3 - Worker Routes Task To Handler

Routing:

```

topicName=sub-lm-put-pending-status
-> PutPendingStatusHandler

```

Handler validates variables:

```

operationId required
subscriptionId required
toStatus required
currentTransitionType required

```

If required variable is missing, fail task with non-retryable error or BPMN error according to the topic contract.

7. Flow 4 - Handler Calls Owing Module API

Example for sub-lm-put-pending-status:

```

PUT /internal/subscriptions/sub_01HZ_NEW/state HTTP/1.1
Authorization: Bearer <subscription-workflow-service-token>
Idempotency-Key: subop_01HZ_NEW_ACT:sub-lm-put-pending-status
Content-Type: application/json

{
  "toStatus": "PENDING_ACTIVATION",
  "currentTransitionType": "ACTIVATE",
  "currentTransitionReasonCode": "SALES_FIELD",
  "operationId": "subop_01HZ_NEW_ACT"
}

```

Rules:

- Use operationId:topicName idempotency key.
- Call owing module API/client.
- Do not write another module database directly.
- Do not branch by operator for business policy.

8. Flow 5 - Worker Completes Camunda Task

If downstream call succeeds:

```

POST /engine-rest/external-task/ext_01HZ_PENDING/complete HTTP/1.1
Content-Type: application/json

{
  "workerId": "subscription-state-worker-01",
  "variables": {
    "priorStatusSnapshot": {

```

```

        "value": "{\\"statusCode\\":\\"CREATED\\"}",
        "type": "Json"
    },
    "lmEventId": {
        "value": "evt_01HZ_PENDING_ACT",
        "type": "String"
    }
}
}
}

```

Only documented output variables should be returned.

9. Flow 6 - Worker Handles Failure

Retryable technical failure:

```

POST /engine-rest/external-task/ext_01HZ_PENDING/failure
Content-Type: application/json

{
    "workerId": "subscription-state-worker-01",
    "errorMessage": "SUB-LM timeout",
    "retries": 2,
    "retryTimeout": 30000
}

```

Business error handled by BPMN:

```

POST /engine-rest/external-task/ext_01HZ_PENDING/bpmnError
Content-Type: application/json

{
    "workerId": "subscription-state-worker-01",
    "errorCode": "INVALID_STATE_FOR_ACTIVATION",
    "errorMessage": "Subscription must be CREATED."
}

```

Use failure for transient technical issues. Use BPMN error for business routes the process model handles.

10. Flow 7 - Message Correlator Receives External Event

Example: BIL publishes payment event.

Message correlator consumes event:

```

{
    "eventType": "BillableEventPaid",
    "payload": {
        "billableEventId": "bevt_01HZ_ACT_001",
        "paidAt": "2026-05-18T10:03:00Z",
        "paidAmount": 3850.00
    }
}

```

Correlator calls Camunda:

```

POST /engine-rest/message
Content-Type: application/json

{
    "messageName": "payment-confirmed",
    "correlationKeys": {
        "billableEventId": {
            "value": "bevt_01HZ_ACT_001",
            "type": "String"
        }
    },
    "processVariables": {
        "paidAt": {"value": "2026-05-18T10:03:00Z", "type": "String"},
        "paidAmount": {"value": 3850.00, "type": "Double"}
    }
}

```

If Camunda has no waiting execution:

1. Store unmatched message.
2. Retry later.
3. Alert if stale.

11. Flow 8 - Metrics Are Emitted

Every task execution emits:

```
worker_task_started_total  
worker_task_completed_total  
worker_task_failed_total  
worker_task_duration_ms
```

Labels:

```
worker_app  
topic_name  
operation_kind  
operator_code  
result
```

Consolidated worker apps must still expose per-topic metrics.

12. Manual Smoke Test

1. Start Camunda.
2. Deploy BPMN with one service task topic.
3. Start worker app subscribed to that topic.
4. Start process instance.
5. Confirm worker fetches task.
6. Confirm downstream API is called.
7. Confirm task completes.
8. Stop worker app and confirm Camunda lock expires.
9. Restart worker and confirm retry/recovery.

13. Common Mistakes

- Treating worker topic as a deployment name.
- No per-topic metrics after consolidation.
- Completing task when downstream call failed.
- Missing idempotency key on downstream call.
- Dropping unmatched messages.
- Putting operator-specific policy inside worker handler.